

Peachy: Intelligent Mobile Calendar with Shared Scheduling and Messaging

Team: Yibing Ju (1009382829), Cynthia Zhou (1009028918), Jonathan Chen (1008849310), Mark Noge (1008992299)

Abstract

Peachy is an intelligent mobile calendar that unifies natural-language scheduling, shared calendars, and in-app messaging into a single workflow. The system combines a React Native (Expo) client with a serverless AWS backend (Lambda, API Gateway, DynamoDB, Cognito, S3) and a hybrid AI pipeline using LLM tool-calling for intent parsing and Thompson sampling for time-slot preference learning. The current implementation includes full mobile UI flows, a typed API client, Cognito-based auth, push notifications, and a CDK-defined backend with calendar, event, chat, pending-invite, friends, RL-preferences, and transcription endpoints. This report covers the problem, design, experiments, and results.

1. Introduction and Problem Statement

Scheduling with others is harder than it should be. Group chats fill up with availability questions, calendars don't share data, and converting "we should meet" into an actual event requires coordination most users lack capacity for. Existing tools address fragments of this problem but not the whole: dedicated calendar apps have no group coordination, scheduling tools require per-event setup, and AI assistants operate in isolation from social context.

Peachy addresses this by unifying calendar management, shared scheduling, and chat, with AI-supported natural-language input. The core proposition: a user says "*dinner with Jordan tomorrow*" and the system checks everyone's availability, picks a suitable time using learned preferences, creates the event, and sends invites — end to end. The goal is to reduce multi-step coordination to a single sentence while keeping users' social graph and preferences private and local to the app.

2. Related Work

2.1 Academic

Peachy's AI pipeline builds on three lines of work. LLMs with tool-calling (Brown et al., 2020; Wei et al., 2022) enable structured intent extraction from free-form text. Thompson sampling (Sutton & Barto, 2018) provides a principled exploration-exploitation framework for learning per-user time-slot preferences without requiring explicit ratings. Conversational scheduling systems (Young et al., 2013) motivate the deterministic post-processing layer that corrects LLM output rather than relying on multi-turn dialog to resolve errors.

2.2 Industry

The team analyzed twelve existing products to identify the gap Peachy fills:

Category	Tools	Key Gap
Availability sharing	When2meet, Doodle, crab.fit	Binary availability, new link per event, no calendar integration
Meeting automation	Calendly, Cal.com	Enterprise-focused; no friend-based social planning
Shared calendars	TimeTree (70M users), Apple/Google/Outlook	No AI; minimal social coordination
AI scheduling	Reclaim.ai , Clockwise	Individual productivity only; no shared social scheduling
Social scheduling	Rodeo	Waitlist; dating-app-adjacent targeting

Summary: Tools fall into two extremes — complex enterprise platforms and lightweight availability polls. AI-driven tools optimize individual schedules, not group coordination. None combine shared calendars, integrated chat, and natural-language scheduling in a mobile-first, social context.

2.3 Datasets

For AI evaluation, the team assessed five aspirational dataset types (NL-to-event pairs, longitudinal schedule patterns, event taxonomy, conflict resolution, completion history) and seven real public datasets (Enron emails, ATUS, Microsoft BA-Calendar, Google DeepMind Natural Plan, KVRET, [Meetup.com](#) events, OpenStreetMap POI). None fully match Peachy's input/output schema; all require significant preprocessing.

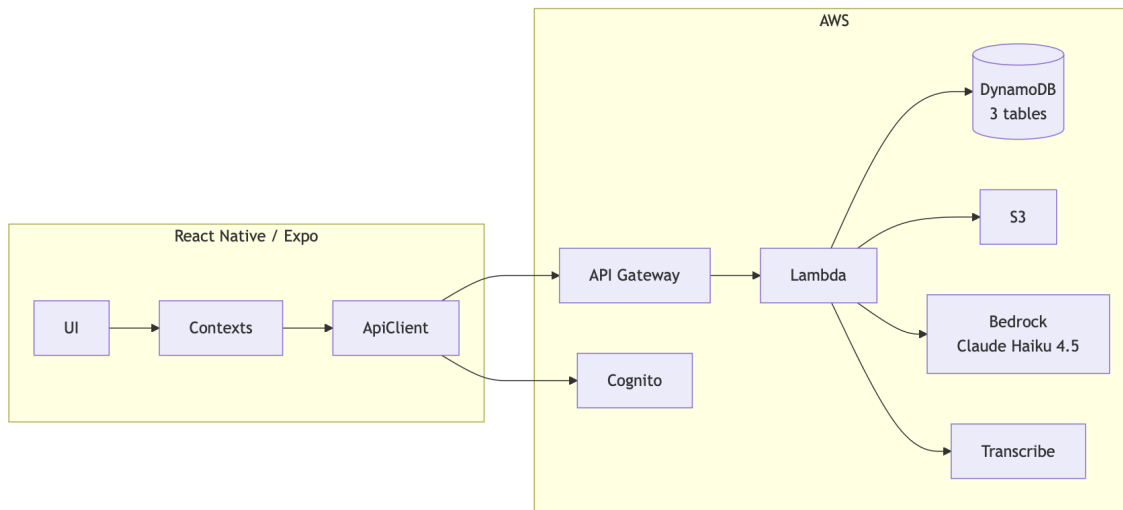
Decision: construct a small internal evaluation dataset from real app usage.

3. Technical Approach, Solution, and Design

3.1 System Architecture

Client: the mobile client is built with React Native (Expo SDK 54), Expo Router, and TypeScript. The app includes day/week/month calendar views, event create/edit/detail modals, shared calendar management, messaging, and an AI input bar. State is managed through React Context providers for authentication, calendars, friends, and chat. JWT tokens are stored in SecureStore, and a typed API client enforces automatic logout on 401 responses. Push notifications are handled with `expo-notifications`.

Backend: The backend is serverless and defined in AWS CDK (TypeScript). The infrastructure is organized into seven stacks: Auth (Cognito with Google OAuth), Database (DynamoDB), Storage (S3), Notifications (SES domain identity provisioned for future transactional email), AI (Bedrock invocation role and policies), API (REST API Gateway with 31+ Lambda handlers), and Monitoring (CloudWatch and AWS Budgets).



3.2 Data Model

Three DynamoDB tables with single-table design for related entities:

- **PeachyMain** — Calendars, Members, Events, Chats, Pending Items. Three GSIs: User Access (GSI1), Calendar Cleanup (GSI2), Date Range (GSI3). TTL on pending items (30-day auto-cleanup).
- **PeachyUsers** — User profiles, RL weights, WebSocket connections. GSIs for username, email, and connectionId lookup. TTL on connections (2-hour cleanup).
- **PeachyMessages** — Chat history. No GSI by design (append-only, saves 50% write cost). All queries on `PK=CHAT#{chatId}`, SK begins_ with `MSG#`.

PITR enabled in production; `RemovalPolicy.RETAIN` on all data-bearing resources so tables, S3 bucket, and Cognito user pool survive `cdk destroy`.

3.3 AI Scheduling Pipeline

A `analyzeInput()` decision function routes each request to one of two paths:

Condition	Path
-----------	------

Exactly 1 explicit time (no @mention)	Fast — Haiku model, caller's own calendar + RL fetch only (no participant lookup)
Multiple times offered, or planning keyword, or @mention, or ambiguous day reference	Smart — Sonnet model, fetch all participants' availability + RL preferences

Smart path (7 steps):

1. Extract @mentions from user input.
2. Resolve date scope with `parseDateHint()`.
3. Execute parallel DynamoDB reads: resolve mentions via username index, load caller calendars, and fetch RL weights by direct PK/SK lookup.
4. Filter resolved mentions by friendship relationship.
5. Fetch participant events using the Date Range index with projected fields (StartTime, EndTime).
6. Compute free windows using merged busy intervals over 1-hour slots.
7. Run `thompsonSampleAll()` across 168 weekly slots and pass top candidate hours to Bedrock with availability context.

Bedrock integration: Models `us.anthropic.claude-haiku-4-5-20251001-v1:0` (Fast path) and `us.anthropic.claude-sonnet-4-5-20250929-v1:0` (Smart path). Forced tool use (`tool_choice: { type: 'tool', name: 'parse_event' }`) guarantees structured JSON output. User input wrapped in `<user_input>` tags for prompt injection mitigation. One retry on throttling/timeout errors.

Post-AI validation (`validateEventTiming`) — 5-step deterministic correction layer:

1. past dates advanced (day-name +7d, same-day past → next hour, catch-all → tomorrow 9 AM)
2. unreasonable hours (23–5) adjusted to 9 AM local, with `EXPLICIT_LATE_NIGHT` exemption
3. end-before-start → default +1 hour
4. explicit "at H:mm am/pm" enforced in local timezone
5. conflict-snap — events moved to nearest free window by local-hour distance if they overlap existing calendar entries.

RL feedback loop: Each slot stores α (accepts) and β (rejections). Accepting an AI-suggested time increments α ; changing it increments β for the AI slot and α for the user's chosen slot.

`AIGenerated`, `AIInput`, and `AIEditedFields` stored on every AI-created event for offline per-field edit-rate analysis.

3.4 Infrastructure and DevOps

laC design principles: One CDK stack per logical service; centralized IAM utilities in `iam.ts` for consistent least-privilege policies; shared Lambda layer (`lambda/shared/`) for common dependencies. Environment config typed in `resources/config.ts`.

Disaster recovery: Because data resources use RETAIN policies, destructive stack operations preserve persistent data. A recovery script discovers orphaned resources, builds CloudFormation import changesets, and re-attaches resources to the stack before recreating stateless infrastructure. The runbook process has been validated in a continuous 37-minute recovery demonstration.

CI/CD: GitHub Actions run on push and pull requests targeting main and dev. The pipeline enforces build and test checks, publishes coverage badges, and posts per-file coverage diffs in pull requests to highlight regressions and improvements.

4. Experiments, Key Metrics, and Results

4.1 Test Coverage

What is tested:

Front-end tests cover core UI flows, context state transitions, component rendering, and date/calendar utilities. Backend tests include Lambda unit tests across domains, CDK assertion tests for all stacks, and handler-level integration tests for complex AI parse behavior. CDK assertion tests synthesize stacks and verify CloudFormation output via `aws-cdk-lib/assertions` without deploying. Lambda unit tests use `aws-sdk-client-mock` to isolate CPU logic from I/O. Integration tests that hit real API Gateway endpoints are run manually and excluded from CI, as are live Bedrock end-to-end tests (`__tests__/integration/ai/`).

What is not tested (by design):

Native Expo push APIs, the HTTP API client, pixel-positioned calendar grid views, AWS Transcribe/Bedrock live calls, and DynamoDB timeout-based retry logic. These require live infrastructure or native device APIs unavailable in Jest.

4.2 Profiling and Optimization

Using Node.js `perf_hooks` with mocked external I/O (DynamoDB: 5–50ms in production; Bedrock: 500–3000ms), the team profiled five Lambda functions to isolate CPU-bound bottlenecks. Mocking at near-zero latency reveals the optimizable work.

Summary of all fixes:

Function	Metric	Before	After	Improvement
<code>parseDateHint</code> (with timezone)	per-call	0.258 ms	0.010ms	96% faster
<code>parseDateHint</code> timezone overhead	ratio vs no-tz	66–74 x	5–6x	~12x better
<code>validateEventTiming</code> (avg)	per-call	0.893 ms	0.095ms	89% faster
<code>validateEventTiming</code> (42 conflict windows)	per-call	3.776 ms	0.230ms	94% faster
<code>updateEvent</code> (in-place)	per-call	1.22ms	0.56ms	54% faster
<code>updateEvent</code> (delete-recreate)	per-call	1.59ms	0.74ms	53% faster
<code>aiParse</code> (smart path, no mentions)	per-call	16.30 ms	3.94ms	76% faster

thompsonSampleAll	structural	full sort 168	topK=5 selection	Correct; noise in timing
userSearch	per-call	0.79ms	—	No change (already efficient)

Root causes and fixes:

- **Fix 1 — Cache Intl.DateTimeFormat instances** (date-utils.ts): `parseDateHint` and `validateEventTiming` are pure functions called on every AI parse request. Every timezone-aware call constructed a new `Intl.DateTimeFormat` — 66–74x slower than no-timezone. A module-level `Map<string, Intl.DateTimeFormat>` cache reduced overhead to 5–6x.
- **Fix 2 — Pre-parse conflict window timestamps** (date-utils.ts): The conflict-snap loop called `new Date()` and `getLocalHour()` per window per iteration. Refactored to pre-compute epoch milliseconds and local hours once before the loop.
- **Fix 3 — Thompson sampling topK + default prior** (thompson.ts): Beta sampling dominates at 89.5% of runtime. Added a pre-allocated `DEFAULT_PRIOR` constant (eliminates 168 object allocations per call for sparse weights) and an optional `topK` parameter using $O(n \times K)$ selection instead of $O(n \log n)$ full sort when callers only need top 5 slots.
- **Fix 4 — Data-driven field mapping loop** (update-event/index.ts): 11 repetitive if-blocks in the update expression builder replaced with a single `UPDATABLE_FIELDS` array and a loop, reducing ~70 lines to ~15.

4.3 Test Depth: AI Parse Pipeline

The AI parse pipeline (`lambda/ai/parse/`) was chosen as the primary deep-testing target — it is the most complex feature (~560 lines, 15+ external calls, two execution paths) and every bug produces events at the wrong time, day, or double-booked.

74 tests across 4 suites: 30 handler-level integration tests (`tests/ai/parse.test.ts` — single-user; `tests/ai/parse-mentions.test.ts` — multi-person), 30 unit tests for `parseDateHint`/`validateEventTiming` with injectable `now` and `timezone`, 14 unit tests for `@mention` resolution and friendship filtering. All use realistic natural language inputs ("soccer on thursday", "dinner with @brian at 8pm") rather than synthetic strings to exercise the same routing users experience.

Test categories: positive (happy path field population), regression (each of the bugs below), negative (bad input, service failure, non-friend exclusion), and edge cases (UTC day-boundary crossing, overlapping multi-participant busy blocks, staggered conflicts).

Bug discovered during testing: `analyzeInput()` checked for an explicit time before checking for @mentions. Inputs like "dinner with @brian at 8pm" had exactly 1 explicit time, triggering the fast-path early return and silently dropping the friend from `invitedUserIds`. Fixed by moving the @mention check above the explicit-time check.

4.4 Model Evaluation

The current AI scheduling pipeline and Bedrock model choices were evaluated using a framework that tests 22 sample use cases across 7 categories. Outputs are scored across 6 weighted dimensions, in addition to other performance metrics including latency and cost.

Category	Workflow path	Number of cases
Explicit time	fast-path	5
Vague date	smart-path	4
Day-name resolution	smart-path	4
Late night	fast-path	3
Conflict snapping	smart-path	2
@Mentions	smart-path	2
Edge cases	N/A	2

Dimension	Weight	Pass condition
dayCorrect	0.25	Local day-of-week matches expectedDay
futureDate	0.20	startTime is after frozenNow
hourInRange	0.20	Local hour within expectedHourRange
endAfterStart	0.15	endTime > startTime
notAdjusted	0.15	validateEventTiming did not need to correct the output (inverted when expectAdjusted: true)
titleNonEmpty	0.05	title.trim().length > 0

Vertical testing revealed that upgrading model families (fast path: Haiku to Sonnet, smart path: Sonnet to Opus) provided similar output quality (-0.01 in cumulative score) while resulting in >118% increase in latency and incurring >76% increase in cost (see Appendix A).

We also compared the production prompt against a 2-shot prompt variant (nshot) using the same 22-case suite (2026-03-30). The nshot variant improved average score and significantly reduced safety-net corrections, at the cost of higher input tokens and overall cost (see Appendix B).

4.6 Performance Targets

Target	Goal	Status
--------	------	--------

API latency (CRUD)	< 500ms	Monitored through CloudWatch
AI scheduling — simple	< 2s	Monitored through CloudWatch; CPU overhead fully tested (<4ms)
AI scheduling — complex	< 5s	Monitored through CloudWatch; smart-path CPU fully tested (76% reduction)

5. Future Work and Next Steps

- **Backend completion:** Chat polling (5s interval when open), profile photo upload via S3 presigned URLs, remaining invitation management endpoints.
 - **AI pipeline:** Follow-up dialog for ambiguous inputs; A/B test Claude Haiku vs Sonnet using `AiEditedFields` edit-rate signal; evals using established tools such as Langfuse for programmatic comparisons of prompt variants; eval for grouped calendars and other interuser use cases
 - **Offline storage:** Local persistence layer (SQLite or AsyncStorage) with conflict resolution.
 - **Email:** Real domain + DNS (DKIM, SPF, DMARC), SES production access, welcome/reset/invite email templates.
 - **Security:** Migrate Google OAuth secret to Secrets Manager; add Bedrock Guardrails; add CloudWatch alarms (Lambda errors, API Gateway 5xx, DynamoDB throttles).
 - **Accessibility:** WCAG 2.1 AA audit across major flows.
-

6. Team Reflection and Course Connection

Peachy gave the team end-to-end experience across requirements engineering, system design, applied ML, infrastructure, and testing discipline — the full arc of the course.

The most concrete course connection was the data pipeline work. We defined aspirational datasets, assessed real public datasets and their limitations, decided pragmatically to collect internal evaluation data, implemented a live pipeline, profiled it, found bugs, fixed them, and

wrote regression tests. This is the ML engineering lifecycle — not just model selection, but the surrounding validation and iteration infrastructure.

The profiling exercise reinforced a principle that applies broadly: measure before optimizing. The dominant bottleneck was `Intl.DateTimeFormat` construction — a 66–74x timezone overhead that was invisible until we mocked I/O to isolate CPU. Without profiling, we would have optimized the wrong things.

Infrastructure as a first-class artifact was a third key theme. Defining CDK stacks early forced decisions about permissions, data lifecycle, and operational constraints before they became blocking problems. In particular, discussions on the cost and latency of different services with respect to different use cases encouraged us to consider our infrastructure decisions in greater detail. The disaster recovery exercise — destroying and recovering a production environment from RETAIN orphans — is the kind of operational practice rarely addressed in coursework but essential in production systems.

Additionally, course discussion on the importance of LLM evals directly influenced our eval framework, decoupling evaluation from deployment and operationalizing the cost/quality tradeoff explicitly. Distinguishing between correct model output and output bailed out by the safety net allows us to improve our model pipeline with confidence.

Finally, the dark mode bug taught us something about AI-assisted development: Claude produces consistent code only when given consistent context. Updating `CLAUDE.md` with architectural decisions and code patterns reduced inter-developer inconsistency more effectively than code review alone. Maintaining a shared AI context file is a new kind of team practice that emerged directly from this project.

References

- Brown, T. B., et al. (2020). Language models are few-shot learners. *NeurIPS*. <https://arxiv.org/abs/2005.14165>
- Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*. <https://arxiv.org/abs/2201.11903>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). <http://incompleteideas.net/book/the-book-2nd.html>
- Young, S., et al. (2013). POMDP-based statistical spoken dialog systems: A review. *Proceedings of the IEEE*. <https://ieeexplore.ieee.org/document/6592993>

- Google Calendar. <https://calendar.google.com> — Microsoft Outlook Calendar. <https://outlook.microsoft.com> — Apple Calendar. <https://support.apple.com/guide/calendar/welcome/mac>
- Calendly. <https://calendly.com> — Motion. <https://www.usemotion.com> — Reclaim.ai. <https://reclaim.ai> — Clockwise. <https://www.getclockwise.com> — TimeTree. <https://timetreeapp.com> — Cal.com. <https://cal.com>
- Microsoft BA-Calendar Dataset. <https://huggingface.co/datasets/microsoft/ba-calendar>
- Natural Plan (Google DeepMind). <https://github.com/google-deepmind/natural-plan>
- KVRET Dataset. <https://nlp.stanford.edu/blog/a-new-multi-turn-multi-domain-task-oriented-dialogue-dataset/>
- ATUS. <https://www.bls.gov/tus/> — MTUS. <https://www.timeuse.org/mtus>
- Expo. <https://docs.expo.dev> — React Native. <https://reactnative.dev> — AWS CDK. <https://docs.aws.amazon.com/cdk/> — AWS Bedrock. <https://docs.aws.amazon.com/bedrock/>
- Peachy A1 — Landscape Analysis and Project Proposal (2026). Internal course report.
- Peachy A2 — Aspirational Datasets, Data Pipeline, and IaC Implementation (2026). Internal course report.
- Peachy A5 — Profiling, Coverage, and Memorable Bugs (2026). Internal course report.

Appendix A. Evaluation Results for Different Bedrock Model Choices

A. Fast path - Haiku, Smart path - Sonnet

B. Fast path - Sonnet, Smart path - Opus

case	A.score	B.score	Δ score	A.ms	B.ms	Δ ms	A.cost	B.cost	Δ cost
*explicit-1	0.75	0.60	-0.15	3909	6350	+2441	\$0.0032	\$0.0117	+\$0.0085
explicit-2	0.85	0.85	+0.00	3253	5976	+2723	\$0.0036	\$0.0116	+\$0.0080

explicit-3	0.81	0.81	+0.00	3613	7186	+3573	\$0.0038	\$0.0120	+\$0.0081
explicit-4	0.75	0.75	+0.00	2654	5250	+2596	\$0.0034	\$0.0114	+\$0.0081
*explicit-5	1.00	0.85	-0.15	2740	6637	+3897	\$0.0034	\$0.0113	+\$0.0080
vague-1	1.00	1.00	+0.00	7684	6928	-756	\$0.0123	\$0.0178	+\$0.0055
vague-2	1.00	1.00	+0.00	7905	7876	-29	\$0.0124	\$0.0190	+\$0.0065
vague-3	1.00	1.00	+0.00	7405	35851	+28446	\$0.0125	\$0.0185	+\$0.0060
vague-4	1.00	1.00	+0.00	6997	51849	+44852	\$0.0120	\$0.0178	+\$0.0058
daynam e-1	1.00	1.00	+0.00	6790	7054	+264	\$0.0122	\$0.0179	+\$0.0058
daynam e-2	1.00	1.00	+0.00	7223	19912	+12689	\$0.0118	\$0.0176	+\$0.0058
daynam e-3	1.00	1.00	+0.00	6402	5843	-559	\$0.0117	\$0.0172	+\$0.0055

*dayna me-4	0.85	1.00	+0.1 5	731 5	132 76	+596 1	\$0.0 116	\$0.0 164	+\$0.0 048
*latenig ht-1	1.00	0.85	-0.1 5	382 3	646 8	+264 5	\$0.0 042	\$0.0 115	+\$0.0 073
latenigh t-2	1.00	1.00	+0.0 0	567 6	703 5	+135 9	\$0.0 039	\$0.0 119	+\$0.0 080
*latenig ht-3	0.85	0.65	-0.2 0	417 2	686 4	+269 2	\$0.0 042	\$0.0 118	+\$0.0 076
*conflict -1	0.50	0.69	+0.1 9	745 0	210 21	+135 71	\$0.0 118	\$0.0 163	+\$0.0 045
conflict- 2	0.60	0.60	+0.0 0	762 4	724 5	-379	\$0.0 119	\$0.0 140	+\$0.0 021
mentions-1	1.00	1.00	+0.0 0	656 1	308 60	+242 99	\$0.0 115	\$0.0 173	+\$0.0 058
*mentio ns-2	0.73	1.00	+0.2 7	614 2	166 20	+104 78	\$0.0 116	\$0.0 186	+\$0.0 070
*edge-1	1.00	0.73	-0.2 7	149 31	596 5	-896 6	\$0.0 116	\$0.0 176	+\$0.0 060
edge-2	0.80	0.80	+0.0 0	213 5	782 6	+569 1	\$0.0 027	\$0.0 135	+\$0.0 108

MEAN	0.89	0.87	-0.01	6018	13177	+7159	\$0.1873	\$0.3326	+\$0.145
------	------	------	-------	------	-------	-------	----------	----------	----------

Appendix B. N-shot Prompt Evaluation Summary

Metric	baseline	nshot	Δ
Avg score	0.88	0.91	+0.03
Pass rate (≥ 0.80)	16/22 (72.7%)	16/22 (72.7%)	-
<code>validateEventTiming</code> fired	7/22 (31.8%)	3/22 (13.6%)	-18pp
Avg latency	6735ms	5476ms	-1259ms
Avg input tokens	1355	2689	+1334
Total cost (22 cases)	\$0.187	\$0.233	+\$0.046
Monthly projection @ 1000/day	\$255/mo	\$318/mo	+\$63/mo